

Timing Analysis of Shift Register Using OpenSTA

INTRODUCTION:

In this lab, students will learn how to perform static timing analysis on a synthesized Shift register using OpenSTA. They will practice importing the gate-level netlist from Yosys, loading timing constraints, linking standard cell libraries, and generating detailed timing reports. This exercise demonstrates how timing is verified for hardware-efficient logic at the signoff level.

APPLICATION:

A shift register is widely used in digital electronics as a fundamental building block for sequential data storage and manipulation. It consists of cascaded flip-flops that shift bits synchronously on clock edges, enabling serial-to-parallel or parallel-to-serial conversion. Common applications include data buffering in UART communication, LED matrix driving, and ring counters for sequence generation. They are essential in designing delay lines and pattern generators, providing efficient data movement with minimal control logic.

OBJECTIVE:

After completing this lab, you will be able to:

- Import a gate-level netlist and standard cell library into OpenSTA (OSU018 lib).
- Load and apply timing constraints (e.g., clock, input/output delays) using SDC format.
- Perform static timing analysis to check setup and hold margins across the design.
- Understand timing reports, including slack values, critical paths, and violation detection.
- Interpret timing analysis results to guide optimization of digital designs for timing closure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

1.1: Create a directory for your lab: `mkdir <project_name>`

1.2: Open lab directory: `cd <project_name>`

1.3: Create directory for project: `mkdir shift_register`

1.4: Open project directory: `cd shift_register`

Step 2: Create Design file

2.1: Open gvim editor for design with command: `gvim shift_register.v`

RTL Design Code:

```
module shift_register (  
    input wire clk,  
    input wire rst,  
    input wire [7:0] data_in,  
    output reg [7:0] q);  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            q <= 8'b00000000;  
        end else begin  
            q <= {q[6:0], data_in [7]};  
        end  
    end  
endmodule
```

2.2: save the file using command- :wq

Step 3: Create .sdc file

3.1: Open gvim editor for the SDC with command: `gvim shift_register.sdc`

For 10ns:

```
create_clock -period 10 -name clk [get_ports clk]  
set_input_delay 5 -min -rise [get_ports clk] -clock clk  
set_input_delay 5 -max -fall [get_ports {rst data_in}] -clock clk  
set_input_transition 5 -min -rise [get_ports {rst data_in}] -clock clk  
set_input_transition 5 -max -fall [get_ports clk] -clock clk  
set_load -pin_load 4 [get_ports q]  
set_output_delay 2 -min -rise [get_ports q] -clock clk
```

Note: For 5ns, change the clock period to 5ns in .sdc file

Step 4: Static Timing Analysis for the design with OpenSTA

Invoke OpenSTA in the terminal using the command: `sta`

4:1 Use the following commands:

Loads the liberty to osu018 standard cell library

i) `read_liberty osu018_stdcells.lib`

Reads your synthesized gate-level Verilog netlist

ii) `read_verilog shift_register_synth.v`

Links the loaded netlist and library to the top module name

iii) `link_design shift_register`

Loads your timing constraints from an SDC file

iv) `read_sdc shift_register.sdc`

Runs timing analysis to check all timing constraints and generates a report.

v) `report_checks`

OBSERVATION:

- Detect setup and hold timing violations
- Examine input and output delay constraints
- Review total negative slack and worst slack reports

OUTPUT:

Reports:

For 10ns:

→ for setup

```
Startpoint: rst (input port clocked by clk)
Endpoint: _16_ (recovery check against rising-edge clock clk)
Path Group: **asynch_default**
Path Type: max
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00	input external delay
0.00	5.00	rst (in)
3.49	8.49	_09_/Y (INVX1)
0.00	8.49	_16_/R (DFFSR)
	8.49	data arrival time
10.00	10.00	clock clk (rise edge)
0.00	10.00	clock network delay (ideal)
0.00	10.00	clock reconvergence pessimism
	10.00	_16_/CLK (DFFSR)
-0.57	9.43	library recovery time
	9.43	data required time
	9.43	data required time
	-8.49	data arrival time
	0.94	slack (MET)

Startpoint: _16_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _17_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ _16_/CLK (DFFSR)
6.94	6.94	^ _16_/Q (DFFSR)
0.00	6.94	^ _17_/D (DFFSR)
	6.94	data arrival time
10.00	10.00	clock clk (rise edge)
0.00	10.00	clock network delay (ideal)
0.00	10.00	clock reconvergence pessimism
	10.00	^ _17_/CLK (DFFSR)
-1.37	8.63	library setup time
	8.63	data required time
	8.63	data required time
	-6.94	data arrival time
	1.69	slack (MET)

→ [for hold](#)

OpenSTA> report checks -path delay min
Startpoint: rst (input port clocked by clk)
Endpoint: _16_ (removal check against rising-edge clock clk)
Path Group: **async_default**
Path Type: min

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	v input external delay
0.00	0.00	v rst (in)
2.14	2.14	^ _09_/Y (INVX1)
0.00	2.14	^ _16_/R (DFFSR)
	2.14	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ _16_/CLK (DFFSR)
0.40	0.40	library removal time
	0.40	data required time
	0.40	data required time
	-2.14	data arrival time
	1.74	slack (MET)

Startpoint: data_in[7] (input port clocked by clk)
Endpoint: _16_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ input external delay
0.00	0.00	^ data_in[7] (in)
0.00	0.00	^ _16_/D (DFFSR)
	0.00	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00	^ _16_/CLK (DFFSR)
0.38	0.38	library hold time
	0.38	data required time
	0.38	data required time
	-0.00	data arrival time
	-0.38	slack (VIOLATED)

For 5ns:

→ for setup

Endpoint: _16_ (recovery check against rising-edge clock clk)
Path Group: **async_default**
Path Type: max

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00 v	input external delay
0.00	5.00 v	rst (in)
3.49	8.49 ^	_09_/Y (INVX1)
0.00	8.49 ^	_16_/R (DFFSR)
	8.49	data arrival time
5.00	5.00	clock clk (rise edge)
0.00	5.00	clock network delay (ideal)
0.00	5.00	clock reconvergence pessimism
	5.00 ^	_16_/CLK (DFFSR)
-0.57	4.43	library recovery time
	4.43	data required time
		data required time
	-8.49	data arrival time
		slack (VIOLATED)

Startpoint: data_in[7] (input port clocked by clk)
Endpoint: _16_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
5.00	5.00 ^	input external delay
0.00	5.00 ^	data_in[7] (in)
0.00	5.00 ^	_16_/D (DFFSR)
	5.00	data arrival time
5.00	5.00	clock clk (rise edge)
0.00	5.00	clock network delay (ideal)
0.00	5.00	clock reconvergence pessimism
	5.00 ^	_16_/CLK (DFFSR)
-2.86	2.14	library setup time
	2.14	data required time
		data required time
	-5.00	data arrival time
		slack (VIOLATED)

→ for hold

Endpoint: _16_ (removal check against rising-edge clock clk)
Path Group: **async_default**
Path Type: min

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00 v	input external delay
0.00	0.00 v	rst (in)
2.14	2.14 ^	_09_/Y (INVX1)
0.00	2.14 ^	_16_/R (DFFSR)
	2.14	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
	0.00 ^	_16_/CLK (DFFSR)
0.40	0.40	library removal time
	0.40	data required time
		data required time
	-2.14	data arrival time
		slack (MET)

```
Startpoint: data_in[7] (input port clocked by clk)
Endpoint: _16_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min
```

Delay	Time	Description
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	input external delay
0.00	0.00	data_in[7] (in)
0.00	0.00	_16_/D (DFFSR)
	0.00	data arrival time
0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	clock reconvergence pessimism
0.00	0.00	_16_/CLK (DFFSR)
0.38	0.38	library hold time
	0.38	data required time
	0.38	data required time
	-0.00	data arrival time
	-0.38	slack (VIOLATED)

COMPARISION:

Timing	Setup		Hold	
	Path-1	Path-2	Path-1	Path-2
10ns	0.94	1.69	1.74	-0.38
5ns	-4.06	-2.86	1.74	-0.38

Note: Timing can be met by changing constraints in sdc.

CONCLUSION:

OpenSTA verifies the timing of the synthesized shift register design using gate-level netlist and timing constraints. The analysis confirms that all critical paths meet setup and hold requirements, with positive slack values. No timing violations are detected, ensuring reliable operation at the specified clock frequency. The clock, reset, and counter increment logic show correct timing relationships in the reports. Overall, OpenSTA validates that the hardware counter will function accurately in real digital systems.